

AI-AUGMENTED BACKEND DEVELOPMENT
PROJECT REPORT
AI FitTrack API

Title

AI FitTrack API

Description

AI FitTrack API is an AI-Augmented backend Web Application developed using MongoDB, Express.js, and Node.js, integrated with Artificial Intelligence capabilities such as Large Language Models (LLMs), semantic search, vector search, and intelligent recommendations. The system enables users to manage fitness-related information, interact with an AI fitness assistant, receive AI-generated insights, perform natural-language searches, upload and analyze documents, and maintain chat history. By combining traditional backend development with modern AI technologies, the platform provides an intelligent, scalable, and user-friendly fitness management experience.

Scenario

Problem:

Many users maintain fitness information such as daily steps, calories, sleep, water intake, workout duration, personal goals, and fitness documents in different places. Traditional fitness applications may provide stored data and fixed reports, but users may still find it difficult to understand their information through natural language or obtain useful explanations from their own documents. Manual searching and reviewing fitness information can also take time.

AI Solution:

AI FitTrack API solves this problem by combining a standard backend API with AI capabilities. Users can securely register and log in, store fitness records, upload documents, ask questions through an AI assistant, summarize information, and search content using natural language. Embeddings and vector search can be used to find semantically relevant information, while an LLM can generate responses from the retrieved context.

Impact:

The application benefits users who want a centralized way to manage fitness information and interact with it intelligently. Administrators can monitor users and application activity. The system improves accessibility by allowing natural-language interaction and supports future expansion into personalized recommendations and AI-powered insights.

Objectives

- Provide secure user registration and login.
- Manage user fitness data through REST APIs.
- Provide an AI fitness assistant through an API.
- Support document upload and document-based question answering.
- Implement semantic and vector search.
- Store chat history and user activity.
- Provide role-based access for users and administrators.
- Create a scalable MVC-based backend.

System Requirements

Software Requirements:

Software	Requirement / Purpose
Operating System	Windows 10/11, macOS or Linux
Node.js	Version 18 or above; runtime environment
npm	Version 9 or above; package manager
Express.js	Backend framework for REST API development
MongoDB	Database for users, fitness data, documents, chats and logs
MongoDB Atlas Search	Full-text search
MongoDB Atlas Vector Search	Semantic search using embeddings
AI Service	Google Gemini API or another supported LLM provider
LangChain / AI SDK	Optional orchestration layer
Browser	Chrome or Firefox
Postman	API testing
Visual Studio Code	Development environment
Git & GitHub	Version control

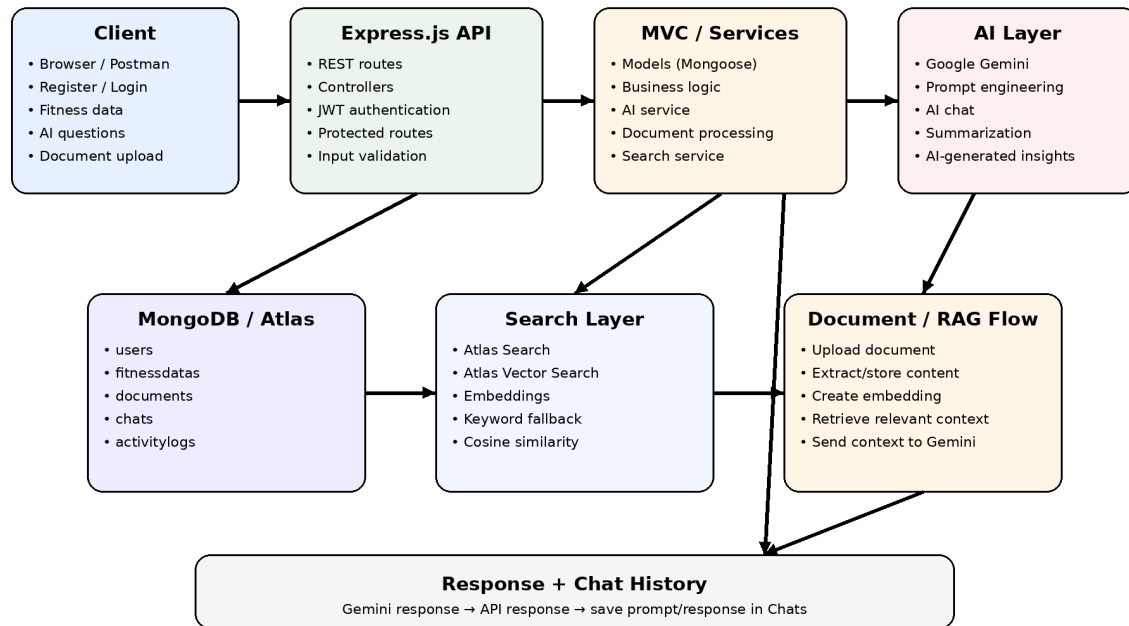
Hardware Requirements:

Component	Requirement
Processor	Intel Core i5 (8th Gen+) / AMD Ryzen 5+
RAM	Minimum 8 GB; recommended 16 GB
Storage	Minimum 5 GB free space
Internet	Required for AI API communication
Display	1366 × 768 or above

Architecture

The AI FitTrack API follows a layered backend architecture. The backend layer contains Node.js, Express.js, REST APIs and JWT authentication. The database layer uses MongoDB, Atlas Search and Atlas Vector Search. The AI layer contains LLM APIs, embedding models, retrieval-augmented generation and prompt engineering. The security layer uses JWT authentication, protected routes, input validation and role-based access control.

AI FitTrack API — Technical Architecture



Architecture Layers

Backend Layer:

- Node.js
- Express.js
- REST APIs

Database Layer:

- MongoDB / MongoDB Atlas
- Atlas Search
- Atlas Vector Search

AI Layer:

- LLM API
- Embedding Model
- RAG Pipeline
- Prompt Engineering
- AI-generated insights

Storage Layer:

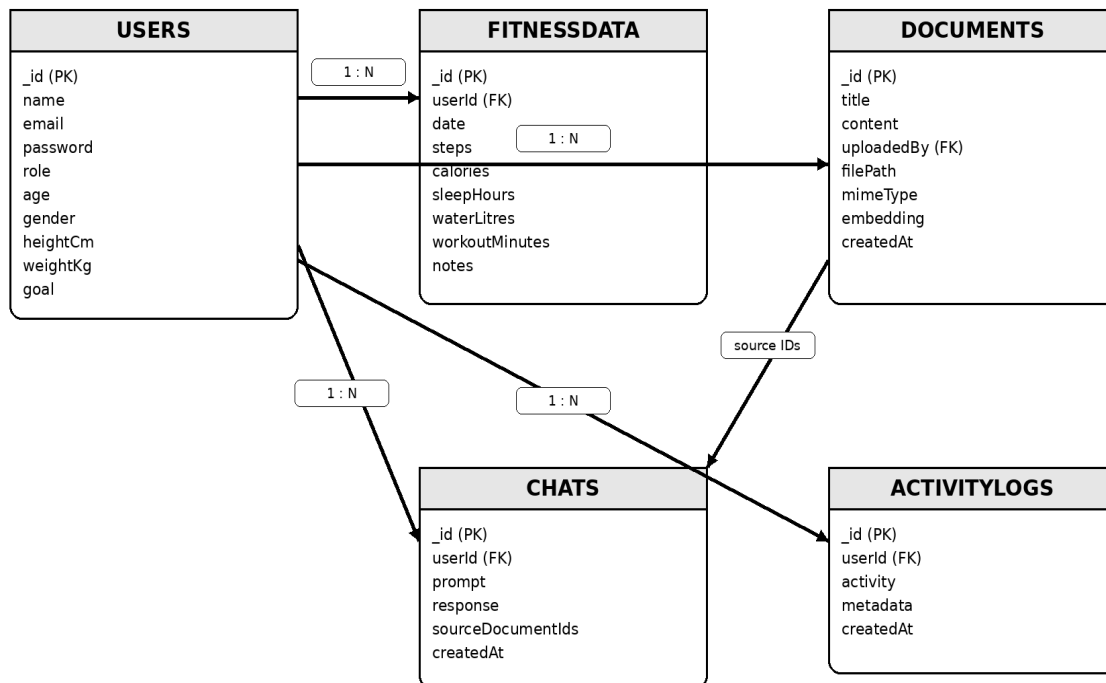
- MongoDB collections
- Uploaded document storage

Security Layer:

- JWT Authentication
- Protected Routes
- Input Validation
- Role-Based Access Control
- Server-side AI API keys.

ER Diagram

AI FitTrack API — ER Diagram



Features

Core Backend Features:

- User Registration
- User Login
- JWT Authentication
- Protected Routes
- Middleware
- CRUD APIs
- Fitness Data Management
- File Upload
- Role-Based Access Control
- Admin Analytics.

AI Features:

- AI Chat Assistant
- Semantic Search
- Vector Search
- Document Question Answering
- Text Summarization
- Content Generation
- Retrieval-Augmented Generation (RAG)
- Intelligent Insights
- Natural Language Queries.

Roles and Responsibilities

Admin:

- Manage users
- Monitor AI usage
- View analytics
- Configure system

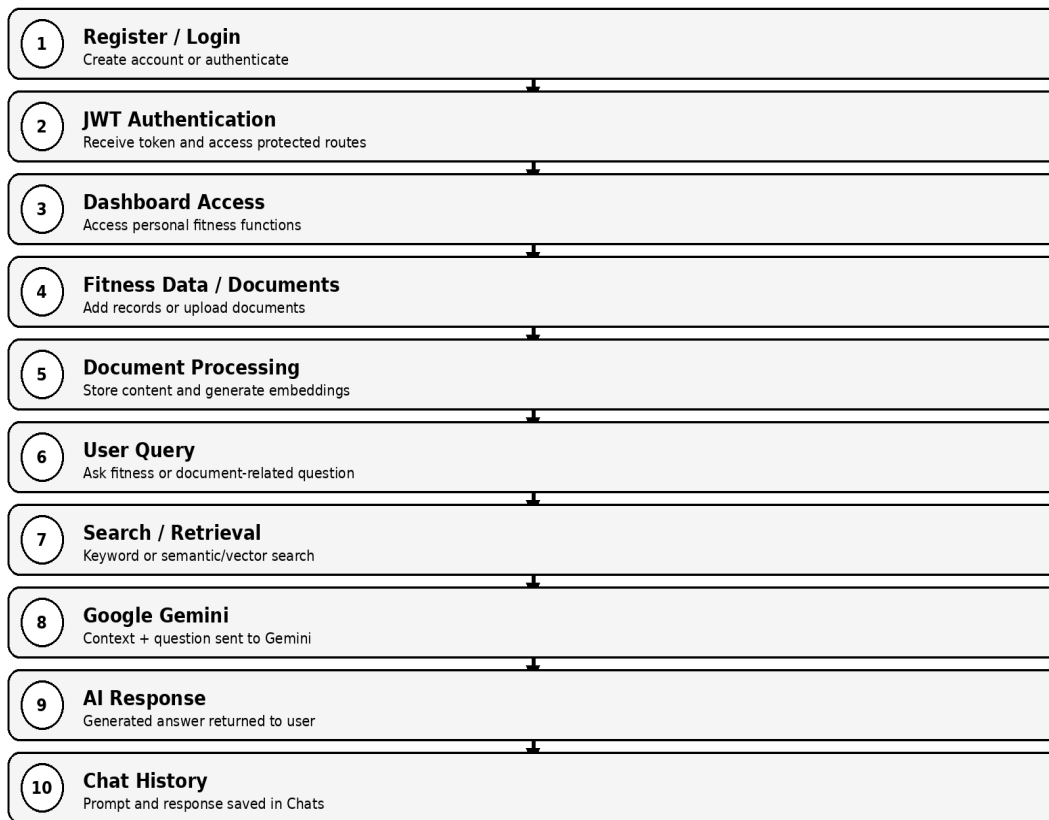
User:

- Register / Login
- Manage profile
- Create fitness records
- Upload documents
- Ask questions
- Search content
- View AI responses
- Manage personal data

User Flow

- User Registration
- User Login
- JWT Authentication
- Dashboard Access
- Enter Fitness Data / Upload Documents
- Generate Embeddings for supported document content
- Store vectors in MongoDB
- Submit a natural-language query
- Execute semantic or keyword search
- Generate AI response
- Display result
- Store chat history

AI FitTrack API – User Flow



MVC Pattern

The backend follows the MVC (Model-View-Controller) architecture. This separates database models, business logic and API routes, making the application easier to maintain, test and extend.

Model Layer:

The Model layer is responsible for database schemas and data access. Mongoose is used to define the models.

- User Schema
- FitnessData Schema
- Document Schema
- Chat Schema
- ActivityLog Schema

Controller Layer:

The Controller layer handles application logic and connects API requests with services and database operations.

- Authentication Logic
- AI API Integration
- Fitness Data Processing
- Document Processing
- Embedding Generation
- Search Operations
- CRUD Operations
- Admin Analytics

Routes Layer:

In this backend API, routes act as the interface through which clients access the controller operations.

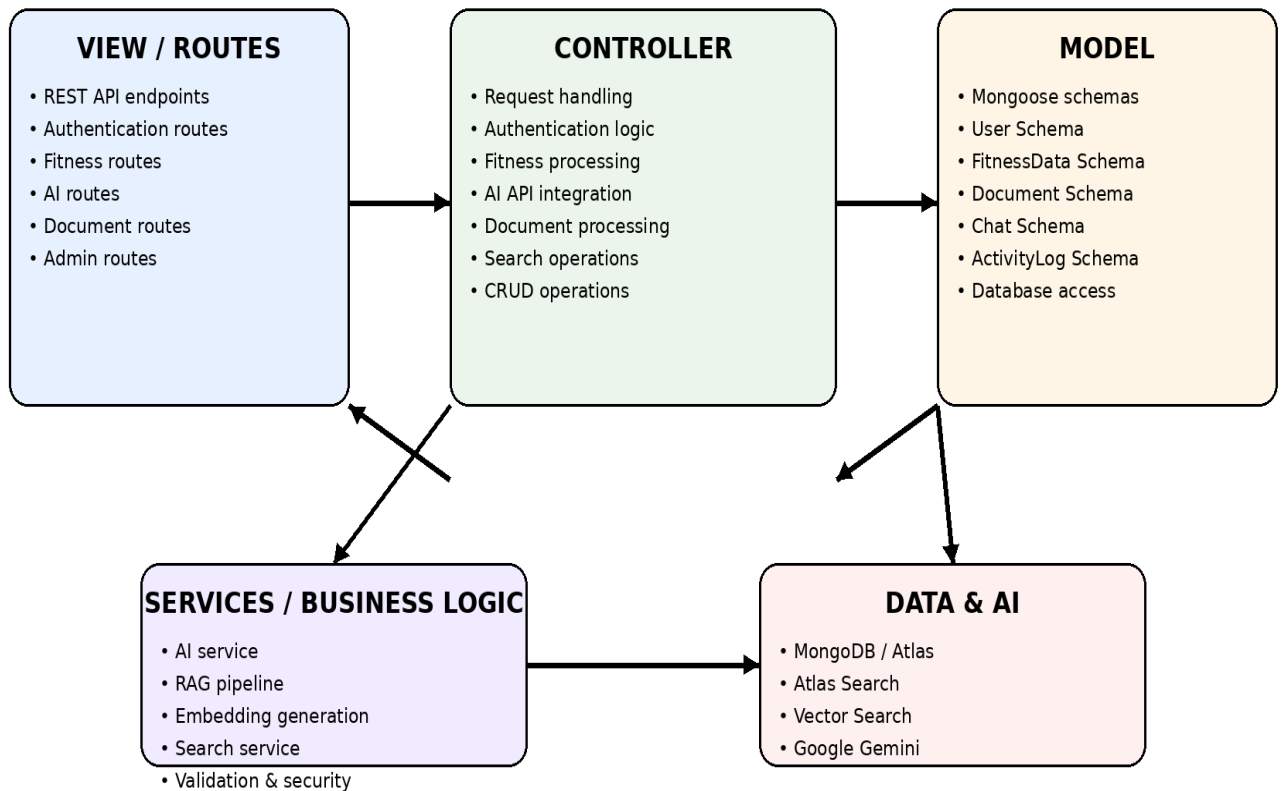
Route Group	Endpoints
Authentication	POST /api/auth/register, POST /api/auth/login
Users	GET/PUT /api/users/profile, GET /api/users/dashboard
Fitness	POST/GET /api/fitness, GET/PUT/DELETE /api/fitness/:id
AI	POST /api/ai/chat, /summarize, /semantic-search; GET /history
Documents	POST /api/documents/upload; GET/DELETE document routes
Admin	GET /api/admin/analytics

Advantages of MVC:

- Separation of concerns
- Better maintainability
- Easy scalability
- Reusable components
- Team collaboration
- Easier testing

MVC ARCHITECTURE PATTERN

AI FitTrack API – MVC Architecture Pattern



Project Setup and Configuration

Project Structure:

```
AI-FitTrack-API/  
├── server.js  
├── config/  
├── models/  
├── controllers/  
├── routes/  
├── middlewares/  
├── services/  
├── uploads/  
├── package.json  
└── .env.example
```

Server Setup:

```
mkdir AI-FitTrack-API
cd AI-FitTrack-API
npm init -y
```

Install Backend Packages:

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken @google/generative-ai multer
npm install nodemon --save-dev
```

Backend API Development

The backend includes authentication APIs, JWT implementation, AI service integration, vector search APIs, CRUD APIs, document upload APIs, fitness APIs and chat APIs.

Authentication API

Registration creates a user account and securely hashes the password using bcrypt. Login validates the credentials and returns a JWT token. Protected routes require the token in the Authorization header.

```
Authorization: Bearer <JWT_TOKEN>
```

Fitness CRUD API

Users can create, read, update and delete their own fitness records. Each record is linked to the authenticated user's ID.

AI API Integration

AI Chat:

The AI chat endpoint accepts a user prompt. The backend can add relevant user fitness context and document search results before sending the request to the LLM. The generated answer is returned to the client and saved in chat history.

Request Structure:

```
POST /api/ai/chat
Authorization: Bearer <token>
Content-Type: application/json

{
  "prompt": "How can I improve my daily fitness routine?"
}
```

Response Structure:

```
{
  "success": true,
  "answer": "AI-generated response..."
}
```

```
"chatId": "CHAT_ID",
"sources": []
}
```

Text Summarization:

POST /api/ai/summarize

```
{
  "text": "Fitness information to summarize..."
}
```

Semantic Search:

The semantic search endpoint converts a query into an embedding and searches for semantically related document content. When an AI key is not configured, the implementation uses a keyword-search fallback.

POST /api/ai/semantic-search

```
{
  "query": "sleep and recovery"
}
```

AI API Cost Considerations:

- LLM API calls may have usage-based costs.
- Embedding generation also consumes AI API usage.
- Limit unnecessary requests and document sizes.
- Use appropriate models for the required task.
- Monitor API usage in production.

Security Practices:

- Keep API keys in .env and never expose them to clients.
- Use HTTPS in production.
- Validate user input.
- Protect AI routes using JWT.
- Apply rate limiting before production deployment.
- Avoid storing unnecessary sensitive information.

Database Development

MongoDB Collections:

Collection	Purpose
users	Stores user accounts, roles and fitness profile information
fitnessdatas	Stores daily fitness records
documents	Stores uploaded document metadata and content
chats	Stores prompts and AI responses
activitylogs	Stores user activity

MongoDB Atlas Search

MongoDB Atlas Search can provide full-text search over document title and content. The project also contains a keyword fallback so basic search can work during local development.

MongoDB Atlas Vector Search

Vector Search supports semantic retrieval. The application stores an embedding array with each supported document. An Atlas Vector Search index can then compare the query embedding with stored document embeddings using cosine similarity.

Suggested Vector Index:

```
{
  "fields": [
    {
      "type": "vector",
      "path": "embedding",
      "numDimensions": 1536,
      "similarity": "cosine"
    }
  ]
}
```

The vector dimension must match the embedding model used by the application.

Sample Fitness :

```
{
  "steps": 8500,
  "calories": 2100,
  "sleep Hours": 7.5,
  "waterLitres": 2.2,
  "workoutMinutes": 45,
  "notes": "Evening walking and strength workout"
}
```

RAG Pipeline

Retrieval-Augmented Generation (RAG) allows the application to retrieve relevant information before asking the AI model to generate an answer.

```
User Question
|
v
Create Query Embedding
|
v
Vector / Semantic Search
|
v
Retrieve Relevant Documents
|
v
Build Context
|
v
Send Context + Question to LLM
|
v
Generate AI Response
|
v
Return Response + Save Chat
```

Prompt Engineering

Prompt engineering is used to guide the AI model toward useful, safe and consistent responses. The AI FitTrack system provides the model with a role, relevant application context and the user's question.

Example :

```
System:
You are AI FitTrack, a fitness information assistant.
Give general wellness and fitness information.
Do not provide diagnosis or emergency medical advice.
```

```
Context:
User goal: ...
Relevant fitness records: ...
Relevant documents: ...
```

```
User:
How can I improve my routine?
```

Document Processing

Users can upload supported files through the document API. Text content can be stored in MongoDB and converted into embeddings. These embeddings are used for semantic retrieval. For production use, document type validation, malware scanning and robust text extraction should be added.

API Testing

Postman can be used to test every REST endpoint. The recommended sequence is registration, login, token setup, fitness CRUD, document upload, AI chat, semantic search and chat history.

Project Execution

Step 1: Install Dependencies

```
npm install
```

Step 2: Configure Environment Variables

```
PORT=8000
MONGO_URI=mongodb://127.0.0.1:27017/ai_fittrack
JWT_SECRET=your_long_secret
GEMINI_API_KEY=your_api_key
GEMINI_MODEL=gemini-1.5-flash
UPLOAD_DIR=uploads
MAX_FILE_SIZE_MB=5
```

Step 3: Run Backend

```
npm run dev
```

Backend URL: http://localhost:8000

API Test Cases

Test	Method	Expected Result
Health Check	GET /api/health	API health message
Register	POST /api/auth/register	User + JWT
Login	POST /api/auth/login	JWT token
Profile	GET /api/users/profile	Authenticated user profile
Create Fitness	POST /api/fitness	Fitness record created
List Fitness	GET /api/fitness	User fitness records
AI Chat	POST /api/ai/chat	AI response + chat ID
Upload Document	POST /api/documents/upload	Document stored
Semantic Search	POST /api/ai/semantic-search	Relevant results
Chat History	GET /api/ai/history	Previous chats

Expected Output Screenshots

The final project report should include screenshots for the following, as required by the template:

- API Testing
- LLM Integration
- AI Chat Interface / API response
- Document Upload
- Search Results
- Recommendation / Insight output
- MongoDB collections
- Vector Search configuration

Functional Module Description

Authentication Module:

Provides secure registration and login. Passwords are hashed before storage, and successful login produces a JWT used to access protected APIs.

Fitness Management Module:

Allows users to store daily fitness information such as steps, calories, sleep, water intake, workout duration and notes.

AI Assistant Module:

Provides natural-language interaction with the AI model. The assistant can use relevant user context and retrieved documents to generate a response.

Document Module:

Allows authenticated users to upload documents and store their content. Embeddings can be generated for semantic retrieval.

Search Module:

Provides keyword fallback and vector-based semantic search. Vector search is intended to use MongoDB Atlas Vector Search.

Admin Module:

Provides administrative analytics such as user count, document count, chat count and activity count. The route is protected by role-based access control.

Advantages

- Centralized fitness data management.
- Natural-language interaction.
- AI-assisted information retrieval.
- Semantic document search.
- Secure JWT-based authentication.
- MVC architecture supports maintainability.
- MongoDB provides flexible document storage.
- Can be extended with recommendation and analytics modules.

Limitations and Future Enhancements

- Add production-grade PDF/DOCX text extraction.
- Add advanced Atlas Search index configuration.
- Add rate limiting and audit controls.
- Add a frontend dashboard/mobile client.
- Add charts for fitness trends.
- Add configurable recommendation rules.
- Add stronger document security and malware scanning.
- Add automated tests and CI/CD deployment.

Mandatory Functionalities Checklist

Requirement	Status in Project
MVC Architecture	Implemented
JWT Authentication	Implemented
Semantic Search	Implemented with vector path + keyword fallback
MongoDB Atlas Search	Architecture supported; index configuration required in Atlas
MongoDB Atlas Vector Search	Implemented in search service; Atlas index required
Prompt Engineering	Implemented in AI service
AI API Integration	Implemented with Google Gemini API
Role-Based Access Control	Implemented for admin analytics

Conclusion

AI FitTrack API is an AI-Augmented backend application that combines Node.js, Express.js, MongoDB, JWT authentication and AI services to create an intelligent fitness information platform. The project follows an MVC architecture and supports fitness data management, document processing, AI chat, semantic search, vector search and role-based access. The design provides a foundation that can be extended into a complete fitness application with a web or mobile frontend, advanced analytics and personalized AI recommendations.

Project Links

Demo Video Link: Add project demo URL here.

GitHub Link: Add repository URL here.

API Documentation: Add link here.